One Service, Many Computers

Distributed systems: cooperation across machines

SCAN / LEARN

THE MAIN IDEA

A distributed system is a group of computers that cooperate over a network to provide one service. Coordination is useful, but messages can be late or lost.

FOLLOW THE SYSTEM

01

ONE REQUEST

A person asks for a page or a result.

02

MANY MACHINES

Different computers run parts of the service.

03

MESSAGES

Computers exchange data through the network.

04

ONE RESPONSE

The service combines work and replies.

USER → SERVICE → COMPUTERS → USER

COMMON MISCONCEPTION

More computers always means a faster answer.

MORE ACCURATE

Networks, shared data and coordination can slow the whole service.

THREE QUESTIONS TO KEEP

PROBLEM: How can several computers serve one application?

FAILURE: One delayed part may delay the whole answer.

IN REAL LIFE: Online shops, search, banking and AI.

SOLO DISCOVERY

Draw two servers: what happens if one loses the network?

What If a Server Disappears?

Failures, replicas and recovery

SCAN / LEARN

THE MAIN IDEA

Failures are normal: hardware stops, networks split, software crashes. A resilient service detects trouble and may move requests to a healthy copy.

FOLLOW THE SYSTEM

01

HEALTH CHECK

The system tests if a server responds.

02

FAILURE

One server or network link becomes unavailable.

03

REROUTE

A healthy replica receives new requests.

04

RECOVER

The failed component is repaired and checked.

CHECK → DETECT → SWITCH → RECOVER

COMMON MISCONCEPTION

Having a spare server means nothing can fail.

MORE ACCURATE

Shared power, storage or bad configuration can still break both.

THREE QUESTIONS TO KEEP

PROBLEM: How can a service survive one failed server?

FAILURE: Missing replicas or shared faults cause an outage.

IN REAL LIFE: Cloud applications and online payments.

SOLO DISCOVERY

Draw two servers and one shared database: find the weak point.

Three Copies, Which Is Right?

Replication and consistency

SCAN / LEARN

THE MAIN IDEA

Replication stores copies of data in different places. Copies may temporarily disagree, so the system needs rules for updates, reads and conflicts.

FOLLOW THE SYSTEM

01
ORIGINAL
A customer changes an address.

›

02
PROPAGATE
The update travels to other replicas.

›

03
DELAY
One replica still has the old address.

›

04
RECONCILE
A consistency rule determines what to return.

WRITE → COPY → DELAY → CONSISTENCY RULE

COMMON MISCONCEPTION

Three copies always contain identical data at every instant.

MORE ACCURATE

Some designs allow temporary differences to remain available.

THREE QUESTIONS TO KEEP

PROBLEM: How do copies stay useful after an update?

FAILURE: Stale information produces wrong decisions.

IN REAL LIFE: Cloud databases and global services.

SOLO DISCOVERY

Write two different updates on two sheets. Which wins, and why?

Two Programs, One Number

Race conditions and shared state

SCAN / LEARN

THE MAIN IDEA

A race condition appears when the result depends on which operation happens first. Without coordination, two valid updates can overwrite each other.

FOLLOW THE SYSTEM

01 START

A shared account balance is 10.

02 READ

Two programs each read the value 10.

03 CHANGE

Each independently calculates 10 + 1.

04 OVERWRITE

Both write 11, even though two updates occurred.

READ 10 → TWO UPDATES → WRITE 11 (NOT 12)

COMMON MISCONCEPTION

Each program is correct, so their combined result must be correct.

MORE ACCURATE

Shared updates need safe ordering or atomic operations.

THREE QUESTIONS TO KEEP

PROBLEM: How do simultaneous updates avoid data loss?

FAILURE: A change disappears without any error message.

IN REAL LIFE: Counters, stock levels, account balances.

SOLO DISCOVERY

Act out two readers and two writers on one number.

A Transaction: All or Nothing

Safely combining database changes

SCAN / LEARN

THE MAIN IDEA

A database transaction groups related changes into one unit. If a required step fails, the system can undo the unfinished unit instead of leaving half a result.

FOLLOW THE SYSTEM

01 BEGIN

A customer transfers money.

02 DEBIT

The sender account is reduced.

03 CREDIT

The receiver account is increased.

04 COMMIT / ROLLBACK

Save both changes, or undo the transaction.

BEGIN → UPDATE A → UPDATE B → COMMIT

COMMON MISCONCEPTION

Writing two lines of database code makes the changes inseparable.

MORE ACCURATE

A transaction and correct isolation settings are needed.

THREE QUESTIONS TO KEEP

PROBLEM: How do related updates succeed together?

FAILURE: Half-completed operations corrupt records.

IN REAL LIFE: Bank transfers, orders and reservations.

SOLO DISCOVERY

Invent a two-step booking: what if the second step fails?

The Traffic Director

Load balancing and bottlenecks

SCAN / LEARN

THE MAIN IDEA

A load balancer distributes incoming requests among healthy workers. It can reduce overload but cannot invent unlimited capacity.

FOLLOW THE SYSTEM

01 REQUESTS

Many devices request the same service.

02 BALANCER

The front door selects a healthy worker.

03 WORKERS

Several servers complete the requests.

04 FEEDBACK

Health and load measurements change routing.

VISITORS → BALANCER → HEALTHY WORKERS

COMMON MISCONCEPTION

Load balancing makes a slow database fast automatically.

MORE ACCURATE

A shared bottleneck remains until its own capacity changes.

THREE QUESTIONS TO KEEP

PROBLEM: How do we distribute incoming traffic?

FAILURE: One worker is flooded while others sit idle.

IN REAL LIFE: Shopping, video platforms and AI services.

SOLO DISCOVERY

Route 12 numbered requests to 3 servers using round-robin.

A Program in a Container

Packaging software for predictable runs

SCAN / LEARN

THE MAIN IDEA

A container packages an application with its runtime dependencies. Containers share the host operating-system kernel; they are not automatically separate virtual machines.

FOLLOW THE SYSTEM

01

APPLICATION

A program needs specific libraries.

›

02

IMAGE

Code and dependencies are packaged.

›

03

CONTAINER

The image runs as an isolated process.

›

04

HOST

The host supplies shared kernel and resources.

CODE + LIBRARIES → IMAGE → CONTAINER

COMMON MISCONCEPTION

A container includes an entire independent operating system.

MORE ACCURATE

Isolation is limited; the host kernel is still shared.

THREE QUESTIONS TO KEEP

PROBLEM: How can software run predictably elsewhere?

FAILURE: Missing dependencies or bad permissions break it.

IN REAL LIFE: Services, development and deployments.

SOLO DISCOVERY

List what an app needs beyond its code to start.

Who Runs All Those Containers?

Kubernetes: orchestrating containers

SCAN / LEARN

THE MAIN IDEA

Kubernetes is software that coordinates containers across computers. It observes the desired configuration and tries to keep actual running services close to it.

FOLLOW THE SYSTEM

01 DESIRED STATE

A service should have three running copies.

02 SCHEDULER

Kubernetes places work on suitable machines.

03 OBSERVE

It checks which copies are healthy.

04 RECONCILE

It restarts or replaces missing copies.

DESIRED STATE → PLACE → CHECK → REPAIR

COMMON MISCONCEPTION

Kubernetes guarantees that every application works.

MORE ACCURATE

It manages deployment and recovery, not faulty business logic.

THREE QUESTIONS TO KEEP

PROBLEM: How do we coordinate many containers?

FAILURE: Bad settings may restart the wrong things.

IN REAL LIFE: Cloud clusters and large applications.

SOLO DISCOVERY

Draw 3 machines and place 5 tasks with limited space.

A Cache Shared by Many Servers

Distributed caching and stale data

SCAN / LEARN

THE MAIN IDEA

A cache keeps reusable results close to where they are needed. A shared cache can help a server fleet, but old or incorrect entries can spread mistakes.

FOLLOW THE SYSTEM

01 ASK

An application requests a stored result.

02 LOOKUP

The cache checks whether it has a valid copy.

03 HIT / MISS

Reuse the entry, or ask the original source.

04 REFRESH

A policy expires or updates old entries.

REQUEST → CACHE → REUSE OR FETCH → REFRESH

COMMON MISCONCEPTION

Caching every answer is always safe and cheaper.

MORE ACCURATE

Freshness, privacy and storage costs determine safe reuse.

THREE QUESTIONS TO KEEP

PROBLEM: How can repeated reads avoid repeated work?

FAILURE: An old cached price misleads a customer.

IN REAL LIFE: Web apps, databases and AI workflows.

SOLO DISCOVERY

Write 3 changing facts. Which are safe to reuse tomorrow?

Why Is the Network Slow?

Latency, bandwidth and waiting

SCAN / LEARN

THE MAIN IDEA

Latency is the delay before data arrives. Bandwidth is how much data can travel per second. More bandwidth does not remove every delay.

FOLLOW THE SYSTEM

01 REQUEST
A tiny message leaves your device.

›

02 TRAVEL
The network adds transmission delays.

›

03 WAIT
Servers queue the request while busy.

›

04 RESPONSE
Processing and return travel take time.

LATENCY = TRAVEL + WAITING + PROCESSING

COMMON MISCONCEPTION

A fast internet plan makes every remote service instant.

MORE ACCURATE

Distance, congestion and server work still take time.

THREE QUESTIONS TO KEEP

PROBLEM: Why is one request slower than another?

FAILURE: A small delay repeated many times feels huge.

IN REAL LIFE: Gaming, video calls and remote AI.

SOLO DISCOVERY

Compare downloading a large file with waiting on a tiny answer.

How Do Engineers See the Invisible?

Logs, metrics and traces

SCAN / LEARN

THE MAIN IDEA

Observability means learning what a system does from its signals. Logs record events, metrics show numbers over time, and traces follow a request through services.

FOLLOW THE SYSTEM

01

EVENT LOG

A message says what happened and when.

02

METRIC

A chart shows errors or response times.

03

TRACE

A request gets tracked across services.

04

DIAGNOSIS

Signals together point to a likely cause.

EVENTS + NUMBERS + PATHS → DIAGNOSIS

COMMON MISCONCEPTION

One dashboard graph tells the entire story.

MORE ACCURATE

A graph can reveal symptoms; traces and logs help find causes.

THREE QUESTIONS TO KEEP

PROBLEM: How can we investigate a hidden slowdown?

FAILURE: Without signals, incidents are guesswork.

IN REAL LIFE: Site reliability and AI serving.

SOLO DISCOVERY

Invent a slow page. What log, metric and trace would help?

Faster, Cheaper, or Safer?

The engineering trade-off triangle

SCAN / LEARN

THE MAIN IDEA

Every design trades cost, speed and reliability. Optimization starts with a measurable goal, then a test of both benefits and side effects.

FOLLOW THE SYSTEM

01 MEASURE
Record speed, errors and resource cost.

02 CHOOSE
Select one bottleneck worth improving.

03 CHANGE
Try a cache, smaller worker or better route.

04 COMPARE
Measure the new result and its risks.

MEASURE → CHANGE → MEASURE AGAIN → DECIDE

COMMON MISCONCEPTION

The cheapest solution is automatically the most efficient.

MORE ACCURATE

A cheap failure can cost more than a reliable service.

THREE QUESTIONS TO KEEP

PROBLEM: How do we spend fewer resources safely?

FAILURE: Optimizing one number makes another worse.

IN REAL LIFE: Cloud bills, AI inference and streaming.

SOLO DISCOVERY

Compare two designs: one faster but 2x costlier. Choose a goal.